Prompt Engineering Basics — Complete Study Guide

Topic: Zero/One/Few-Shot Prompting, System Prompts, Role Prompts
Estimated Study Time: 6–8 hours
Output Goal: Create 10 prompts for different tasks

BEST VIDEO RESOURCES (Easy & Beginner-Friendly)

Tier 1 — Must Watch

#	Video / Course	Creator	Duration	Link
1	**ChatGPT Prompt Engineering for Developers** -- **[VERY IMPORTANT — Watch This First]**	Andrew Ng + Isa Fulford (OpenAI)	1.5 hrs	DeepLearning.AI (Free)
2	**Prompt Engineering for ChatGPT**	Dr. Jules White, Vanderbilt	18 hrs (self-paced)	Coursera (Free to audit)
3	**Prompt Engineering Fundamentals**	Microsoft (Nitya Narasimhan)	15 min	Microsoft Learn
4	**Understanding Prompt Engineering**	DataCamp	1 hr	DataCamp (Free)

Tier 2 — Hands-On Practice

#	Resource	Creator	Link
5	**Interactive Prompt Engineering Tutorial**	Anthropic	GitHub
6	**Maximizing LLM Performance** (Prompt Eng vs RAG vs Fine-tuning)	OpenAI	YouTube
7	**Generative AI for Everyone**	Andrew Ng	Coursera

Tier 3 — Reference Material

#	Resource	Link
8	Anthropic Prompt Engineering Docs	docs.anthropic.com
9	OpenAI Prompt Engineering Guide	platform.openai.com/docs
10	Learn Prompting (community)	learnprompting.org

#	Resource	Link
11	DAIR.AI Prompt Engineering Guide	promptingguide.ai

STUDY MATERIAL — DETAILED EXPLANATIONS

Module 1: What is Prompt Engineering?

Prompt engineering is the practice of crafting inputs to get useful, accurate, and consistent outputs from a large language model. Think of it as learning to communicate clearly with someone who is extremely knowledgeable but takes everything quite literally.

The exact same LLM can produce wildly different results depending on how you ask:

Vague prompt: `Tell me about Python.` Result: Could be about the language, the snake, or Monty Python. Ambiguous.

Engineered prompt:

```
Explain Python programming to someone with a Java background.
Focus on key syntax differences and Python-specific features
like list comprehensions and duck typing. Keep it under 300 words.
```

Result: A focused, useful comparison tailored to the reader.

The anatomy of a good prompt

Every effective prompt combines some of these components:

- **Instruction** — What to do ("Summarize", "Classify", "Rewrite")
- **Context** — Background info the model needs
- **Input data** — The specific content to work with
- **Output format** — How to structure the result ("JSON", "3 sentences", "a table")

Not every prompt needs all four, but complex tasks benefit from including more.

Module 2: Zero-Shot, One-Shot, and Few-Shot Prompting

The "shot" means the number of examples you provide before asking the model to perform the task.

Zero-Shot Prompting (0 examples)

Give the model instructions with no examples. You rely on its pre-trained knowledge.

When to use: Straightforward tasks — classification, Q&A, summarization, translation.

```
Classify this review as Positive, Negative, or Neutral.

Review: "The delivery was fast but the packaging was damaged.
Product itself works fine."

Classification:
```

Modern instruction-tuned models understand what "classify" means from training. For common tasks, clear instructions are sufficient. It breaks down when tasks are unusual or require very specific output formats.

One-Shot Prompting (1 example)

Provide one input-output example before the actual task.

When to use: When the task is slightly ambiguous or you need a specific format.

```
Extract key details from job postings in a structured format.

Example:
Input: "Senior Data Engineer at CloudTech. 5+ years with
Spark and AWS. $130K-$160K. Hybrid in Seattle."
Output:
- Role: Senior Data Engineer
- Company: CloudTech
- Experience: 5+ years
- Skills: Spark, AWS
- Salary: $130K-$160K
- Location: Seattle (Hybrid)

Now extract from this:
Input: "Marketing Coordinator at BrightWave. 2-3 years in
B2B SaaS. $55K-$70K. Fully remote."
Output:
```

One example is worth a thousand words of instructions. The model picks up field names, ordering, and formatting instantly.

Few-Shot Prompting (2-5 examples)

Provide multiple examples to establish a reliable pattern.

When to use: When consistency matters, when judgment is subjective, or when categories are domain-specific.

```
Classify support tickets by priority.

Ticket: "Can't log in, presentation in 30 minutes."
Priority: URGENT

Ticket: "Dark mode toggle doesn't save between sessions."
Priority: LOW

Ticket: "Entire team lost access to shared dashboard after
today's update."
Priority: HIGH

Ticket: "Could you add CSV export for reports?"
Priority: LOW

Now classify:
Ticket: "Payment processing is completely down. Customers
can't complete purchases."
Priority:
```

Through 4 examples, the model learns your criteria without you having to explain them explicitly.

Quick Comparison

Technique	Examples	Best For	Tradeoff
Zero-shot	0	Simple, well-defined tasks	Cheapest, less format control
One-shot	1	Setting format or style	Good balance
Few-shot	2-5	Consistent classification, complex patterns	Most reliable, uses more tokens

Tips for choosing examples: Be diverse (cover edge cases), be representative (use realistic inputs), stay consistent (same format for each), and put the best examples last (models attend more to nearby content).

Module 3: System Prompts

A system prompt sets the model's overall behavior, personality, and constraints for an entire conversation. Most LLM APIs use a "role" system:

```
System:  "You are a helpful financial advisor who explains
          concepts in plain language. Always remind users
          you are not providing official financial advice."

User:    "Should I invest in index funds?"
```

The **system message** stays constant across the whole conversation. The **user message** changes with each turn. The model uses the system prompt as persistent context for how it should behave.

What to put in a system prompt

A good system prompt typically defines:

1. Identity and role: Who is the model pretending to be?

```
You are a senior Python developer who specializes in clean,
readable code. You follow PEP 8 and prefer simple solutions
over clever ones.
```

2. Behavioral rules: What should it always or never do?

```
Always provide code examples with your explanations.
Never suggest deprecated libraries.
If you're unsure about something, say so rather than guessing.
```

3. Output constraints: How should responses be formatted?

```
Keep answers under 200 words unless the user asks for more detail.
Use code blocks for all code snippets.
End each response with a one-line summary.
```

4. Tone and style:

```
Be direct and concise. Avoid filler phrases like "Great question!"
Use a professional but friendly tone.
```

System prompt example — Customer Support Bot

```
System: You are a customer support agent for TechGadgets, an
online electronics store.

Rules:
- Be polite and empathetic, but concise
- If the customer has a return request, ask for the order number
```

```
- You can offer refunds for orders within 30 days
- For orders older than 30 days, offer store credit only
- Never share internal policies or pricing margins
- If you don't know the answer, say you'll escalate to a
  human agent
- Always end with "Is there anything else I can help with?"
```

In chat interfaces vs API

When using ChatGPT or Claude through the web interface, you can write system-prompt-like instructions at the start of your conversation, or use features like "Custom Instructions" (ChatGPT) or "User Preferences" (Claude). When using the API, system prompts are passed as a separate `system` message in the request.

Module 4: Role Prompting (Persona Pattern)

Role prompting is a technique where you instruct the model to adopt a specific identity, expertise, or perspective. It's one of the most powerful and intuitive ways to shape output quality.

Why does assigning a role change the output?

During training, the model has seen text written by experts in every field — doctors writing medical notes, lawyers writing legal briefs, teachers explaining concepts to students. When you tell the model "You are a senior cardiologist," you are effectively asking it to draw on the patterns of medical text written by specialists, which tends to be more precise, use correct terminology, and follow clinical reasoning patterns.

Basic role prompt structure

```
You are a [specific role] with [relevant qualifications/experience].
Your task is to [what you want them to do].
[Additional constraints or context]
```

Examples of role prompts in action

Without role (generic):

```
What should I eat to be healthier?
```

Output: Generic advice like "eat more vegetables."

With role:

```
You are a registered dietitian with 15 years of experience in
sports nutrition. A 30-year-old who runs 5K three times a week
asks you for meal recommendations to improve recovery. Provide
specific, actionable advice.
```

Output: Targeted advice about protein timing, glycogen replenishment, specific food suggestions with macronutrient reasoning.

More role prompt examples:

```
You are an experienced patent attorney. Review this product
description and identify potential patentability issues.
```

```
You are a kindergarten teacher. Explain how the internet
works to a 5-year-old using only concepts they'd understand
from everyday life.
```

```
You are a code reviewer at a FAANG company known for being
thorough but constructive. Review this Python function and
suggest improvements.
```

Combining roles with other techniques

Role prompting is most powerful when combined with system prompts and few-shot examples:

```
System: You are a senior product manager at a B2B SaaS company.
You write clear, data-informed PRDs (Product Requirement
Documents). You always consider edge cases and include
acceptance criteria.

User: Write a PRD for adding a "bulk export" feature to our
analytics dashboard. Users have been requesting this for
6 months and it's blocking 3 enterprise deals.
```

Module 5: Other Essential Techniques

Chain-of-Thought (CoT) Prompting

Ask the model to think step-by-step before answering. This dramatically improves performance on math, logic, and reasoning tasks.

Zero-shot CoT (just add "think step by step"):

```
A store has 15 apples. They sell 7 in the morning and receive
a shipment of 12 in the afternoon. Then they sell 9 more.
How many apples are left?

Think through this step by step before giving your answer.
```

Few-shot CoT (show the reasoning in your examples):

```
Q: A farmer has 23 sheep. He buys 6 more, then sells 10.
A: Let me work through this.
  Start: 23 sheep
  After buying 6: 23 + 6 = 29
  After selling 10: 29 - 10 = 19
  Answer: 19 sheep

Q: A library has 145 books. They donate 38 and receive 52 new ones.
A:
```

Delimiter/Structured Prompting

Use clear markers to separate different parts of your prompt, especially when working with input text that could be confused with instructions:

```
Summarize the text delimited by triple backticks in one paragraph.

Text: ```[paste article here]```

Summary:
```

Common delimiters include triple backticks, XML tags, or dashes. This prevents "prompt injection" where the input text might contain instructions that confuse the model.

Output Format Specification

Be explicit about the format you want:

```
Extract the following information from the email below and
return it as JSON with these exact keys:
"sender", "subject", "action_items", "deadline"

Email: [paste email]
```

SUGGESTED 6-8 HOUR STUDY PLAN

Time Block	Activity	Duration
Hour 1	Watch: Andrew Ng's Prompt Engineering course (DeepLearning.AI)	1.5h
Hour 2	Watch: Microsoft's Prompt Fundamentals (15min) + Read Module 1-2 of this guide	1h
Hour 3	Hands-on: Open ChatGPT or Claude and practice zero-shot, one-shot, and few-shot prompts yourself	1h
Hour 4	Read: Module 3-4 (System Prompts + Role Prompting) + experiment with system/role prompts	1h
Hour 5	Watch: OpenAI's "Maximizing LLM Performance" talk on YouTube	1h
Hour 6	Work through Anthropic's interactive prompt engineering tutorial (first 5 chapters)	1h
Hour 7	Read: Module 5 (CoT, delimiters, output formats) + practice	0.5h
Hour 7.5-8	Create your 10 prompts for different tasks (see below)	0.5-1h

---

OUTPUT: 10 PROMPTS FOR DIFFERENT TASKS

Here are 10 ready-to-use prompts across various tasks. Study how each one uses the techniques from this guide, then try modifying them for your own needs.

Prompt 1: Email Summarizer (Zero-Shot)

Task: Summarization

```
Read the following email and provide:
1. A one-sentence summary
2. The key action items (if any)
3. The deadline (if mentioned)

Email:
"""
[Paste your email here]
"""
```

Technique used: Zero-shot with structured output format.

Prompt 2: Code Reviewer (Role + System Prompt)

Task: Code Review

```
System: You are a senior software engineer conducting a code
review. You are thorough but kind. You focus on readability,
correctness, performance, and security — in that order.

User: Review this function. Point out issues and suggest fixes.
For each issue, rate its severity as Critical, Warning, or Nit.

def get_user(id):
    user = db.query(f"SELECT * FROM users WHERE id = {id}")
    return user[0]
```

Technique used: Role prompting + system prompt with explicit review criteria.

Prompt 3: Sentiment Classifier (Few-Shot)

Task: Text Classification

```
Classify each product review as Positive, Negative, or Mixed.

Review: "Absolutely love this keyboard. The keys feel great."
Sentiment: Positive

Review: "Terrible battery life. Barely lasts 2 hours."
Sentiment: Negative

Review: "The camera is excellent but the phone overheats often."
Sentiment: Mixed

Review: "Fast shipping, decent quality for the price, though
the color was slightly different from the photos."
Sentiment:
```

Technique used: Few-shot with 3 diverse examples covering all categories.

Prompt 4: Meeting Notes Generator (Zero-Shot + Format)

Task: Summarization / Extraction

```
Convert the following raw meeting transcript into structured
meeting notes using this exact format:

## Meeting Summary
[2-3 sentence overview]

## Key Decisions
- [decision 1]
- [decision 2]

## Action Items
- [ ] [task] — Owner: [name] — Due: [date]

## Open Questions
- [unresolved items]

Transcript:
"""
[Paste transcript here]
"""
```

Technique used: Zero-shot with very explicit output template.

Prompt 5: Technical Explainer (Role + Audience Adaptation)

Task: Explanation / Teaching

```
You are an experienced computer science professor known for
clear analogies. Explain the concept of "API rate limiting"
at three levels:

1. For a 10-year-old (use a real-world analogy)
2. For a business manager (focus on why it matters)
3. For a junior developer (include technical details)

Keep each explanation under 100 words.
```

Technique used: Role prompt + multi-audience approach + word limit.

Prompt 6: Bug Report Writer (One-Shot)

Task: Structured Writing

```
Write a clear bug report from the user's description. Follow
this format exactly:

Example:
Description: "The app crashes when I try to upload a photo"
Bug Report:
**Title:** App crash on photo upload
**Steps to Reproduce:** 1) Open app 2) Tap upload button
3) Select photo from gallery
**Expected:** Photo uploads successfully
**Actual:** App crashes immediately after photo selection
**Severity:** High
**Environment:** Not specified

Now write a bug report for:
Description: "When I change my password, it says success but
the old password still works and the new one doesn't"
Bug Report:
```

Technique used: One-shot with explicit format template.

Prompt 7: Data Analyst (Role + CoT)

Task: Data Analysis

```
You are a data analyst. I'll give you sales data and I want
you to analyze it.

Think through your analysis step by step:
1. First, identify the overall trends
2. Then, spot any anomalies or outliers
3. Finally, provide 3 actionable recommendations

Data:
Q1: $125K | Q2: $142K | Q3: $98K | Q4: $210K
Product A: $300K | Product B: $175K | Product C: $100K
Customer retention: 78% (industry avg: 85%)

Analysis:
```

Technique used: Role prompt + chain-of-thought + structured steps.

Prompt 8: Email Drafter (Role + Constraints)

Task: Writing

```
You are a professional communications specialist. Draft a
polite but firm email to a vendor who has missed their
delivery deadline by two weeks.

Requirements:
- Tone: Professional, not aggressive
- Reference the original deadline (January 15th)
- Request an updated delivery timeline
- Mention that continued delays may affect future contracts
- Keep it under 150 words
- Include a subject line
```

Technique used: Role prompt + detailed constraints for tone and content.

Prompt 9: Study Quiz Generator (Zero-Shot + Format)

Task: Content Creation

```
Create a 5-question quiz on the topic of "prompt engineering
basics" for someone who just finished a beginner course.

Requirements:
- Mix of question types: 2 multiple choice, 2 true/false,
  1 short answer
- Include the correct answer after each question
- Include a brief explanation of why the answer is correct
- Questions should progress from easy to hard
```

Technique used: Zero-shot with detailed format and difficulty specifications.

Prompt 10: Competitive Analysis (Role + Few-Shot + CoT)

Task: Research / Analysis

```
You are a market research analyst. Analyze products by
identifying strengths, weaknesses, and target audience.

Example:
Product: Notion
Strengths: All-in-one workspace, flexible databases,
strong template ecosystem
Weaknesses: Steep learning curve, can be slow with large
databases, limited offline support
Target Audience: Knowledge workers, startups, project teams

Example:
Product: Trello
Strengths: Intuitive Kanban boards, easy onboarding, free
tier is generous
Weaknesses: Limited views, not great for complex projects,
basic reporting
Target Audience: Small teams, freelancers, visual planners

Now analyze:
Product: Slack

Think through the product's features and market position
before responding in the same format.
```

Technique used: Role + few-shot (2 examples) + chain-of-thought instruction.

QUICK REFERENCE — SHORT NOTES

Prompt engineering in one sentence: Crafting inputs to get useful, consistent outputs — by being specific about the task, audience, format, and constraints.

Zero-shot in one sentence: Give instructions only, no examples — works for common, well-defined tasks.

One-shot in one sentence: Give one example to set the format, then ask for the actual task — best for showing style or structure.

Few-shot in one sentence: Give 2-5 examples to teach a pattern, then ask the model to follow it — best for consistent classification and subjective tasks.

System prompt in one sentence: Persistent instructions that define the model's role, rules, and behavior for an entire conversation.

Role prompting in one sentence: Tell the model to act as a specific expert, which activates more specialized and precise knowledge patterns.

Chain-of-thought in one sentence: Ask the model to reason step-by-step before answering, which improves accuracy on logic and math problems.

COMMON MISTAKES TO AVOID

1. Being too vague. "Write something about marketing" gives you garbage. "Write a 200-word LinkedIn post about B2B content marketing trends in 2026, targeting CMOs" gives you gold.

2. Not specifying the audience. The same topic explained to a CEO vs. a junior developer should sound completely different. Tell the model who it's writing for.

3. Ignoring output format. If you don't specify format, the model guesses. Sometimes you get paragraphs when you wanted bullet points, or a novel when you wanted 3 sentences.

4. Cramming too much into one prompt. Break complex tasks into steps. Ask for an outline first, then expand each section — don't ask for a complete research paper in one shot.

5. Not iterating. Your first prompt is rarely perfect. Treat prompt engineering like coding: write, test, refine, repeat.

6. Using the wrong shot count. If zero-shot gives inconsistent results, add examples. If your examples are too similar, the model over-fits to that narrow pattern.

Last updated: February 2026.